

CT2 Question Paper - Question 3

1. Introduction

The journey from writing embedded code to deploying a functional product is fraught with logical errors, timing issues, and hardware-software integration bugs. Debugging is the critical phase where these issues are identified and resolved. As systems grow in complexity—often utilizing Real-Time Operating Systems (RTOS) and object-oriented design—the tools required to understand system behavior must also evolve. This necessitates a clear distinction between low-level hardware interactions (On-board debugging) and high-level software flow (Task-level debugging). Concurrently, modeling system architecture using UML diagrams, specifically Class and Component diagrams, provides the structural blueprint needed to minimize these bugs during the initial design phase.

2. Core Technical Explanation

a) Debugging: On-board vs. Task-Level with Examples

Debugging in embedded systems spans across different layers of abstraction.

On-board Debugging (Hardware/Low-Level): On-board debugging involves inspecting the internal state of the microcontroller hardware while it executes code. It interfaces directly with the silicon CPU core.

- **Mechanism:** It relies on dedicated hardware modules built into the SoC, such as JTAG (Joint Test Action Group) or SWD (Serial Wire Debug). A hardware debugger (e.g., J-Link, ST-Link) connects the host PC to the target board.
- **Capabilities:**
 - Setting hardware breakpoints (halting the CPU when the instruction pointer hits a specific memory address).
 - Single-stepping through assembly or C code.
 - Inspecting and modifying CPU registers and memory-mapped peripheral registers directly.
- **Example:** An engineer writes code to configure a Timer for PWM output, but the motor isn't spinning. Using on-board debugging over SWD, they halt the CPU and inspect the Timer's configuration registers in memory. They discover the "Clock Enable" bit for that specific timer was never set.

Task-Level Debugging (Software/OS-Level): When an RTOS is introduced, the system becomes highly concurrent. On-board debugging falls short here because halting the CPU stops all tasks, breaking the real-time temporal behavior and masking timing bugs like race conditions. Task-level debugging provides visibility into the OS's internal state.

- **Mechanism:** Usually involves software instrumentation, trace macros, or specialized RTOS-aware debugger plugins on the host IDE. It often outputs data via UART, SWO (Serial Wire Output), or Ethernet.
- **Capabilities:**
 - Monitoring task states (Ready, Running, Blocked, Suspended).
 - Analyzing CPU utilization per task.
 - Tracking inter-task communication (IPC) mechanisms like Semaphores, Mutexes, and Message Queues.
 - Detecting stack overflows and memory leaks within specific threads.
- **Example:** A system freezes randomly. The engineer uses task-level trace tools (like Segger SystemView). The trace shows "Task A" takes a Mutex, then gets preempted by "Task B". "Task B" tries to take the same Mutex and blocks. "Task A" is then prevented from running by a medium-priority "Task C". The trace clearly reveals a **Priority Inversion** deadlock, which would be nearly impossible to find by simply stepping through assembly code.

b) Compare and Contrast Class Diagrams and Component Diagrams

Unified Modeling Language (UML) is vital for visualizing system architecture. Class and Component diagrams both describe the static structure of a system, but at different levels of abstraction.

Class Diagrams:

- **Focus:** The logical, micro-level building blocks of the software.
- **Elements:** Classes (representing objects/entities), Attributes (variables/data), Methods (functions/operations).
- **Relationships:** Defines how classes interact through associations, inheritance (generalization), aggregation, and composition.
- **Application:** Used heavily during detailed software design to map out object-oriented code structures before programming begins.

Component Diagrams:

- **Focus:** The physical, macro-level architectural modules and their dependencies.
- **Elements:** Components (encapsulated, reusable pieces of software or hardware, e.g., a shared library, a database, or a sensor driver module), Interfaces (Provided and Required

interfaces represented as "lollipops" and "sockets").

- **Relationships:** Shows dependency relationships; how one component relies on the interface provided by another to function.
- **Application:** Used during high-level system architecture design to plan how large modules integrate, compile, and deploy together.

3. Mathematical / Theoretical Support

While debugging and UML diagrams are less mathematically driven, Task-Level debugging relies heavily on scheduling theory. When analyzing a trace for an RTOS, engineers look for violations of scheduling bounds.

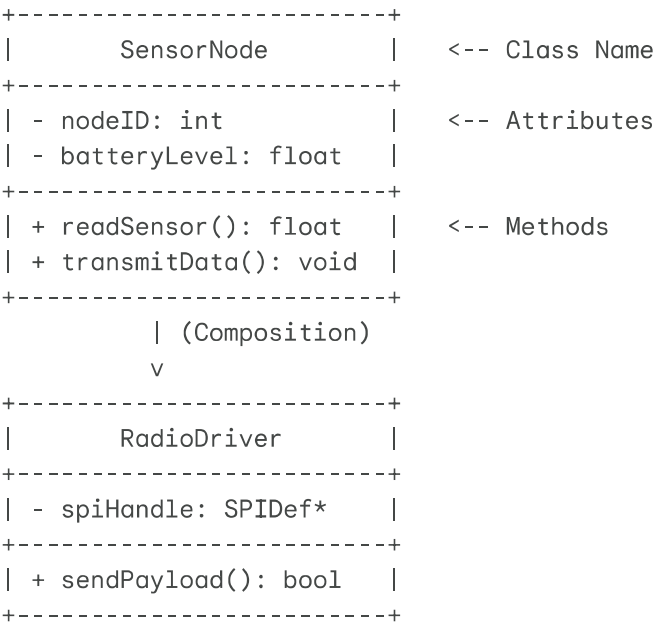
For instance, verifying that the actual measured Worst-Case Execution Time (*WCET*) of a task does not exceed its theoretical bound used during system design:

$$WCET_{actual_measured} \leq C_i$$

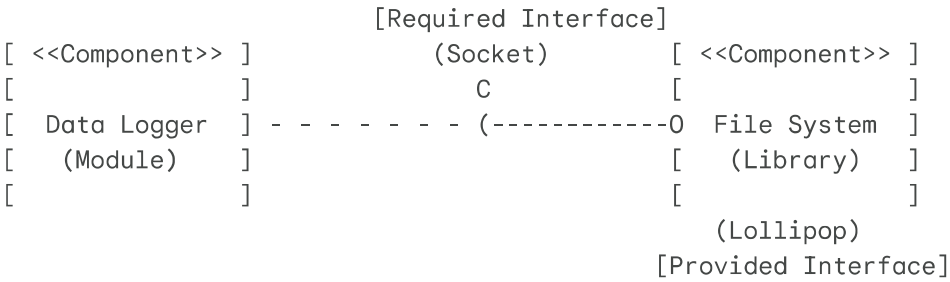
If Task-level debugging shows $WCET_{actual} > C_i$, the system is mathematically prone to deadline misses, indicating a critical need for code optimization.

4. Relatable Diagram: UML Structural Modeling Comparison

Class Diagram (Micro-level detail):



Component Diagram (Macro-level architecture):



5. Industrial Case Study / Real-World Example

Developing a Smart Thermostat:

- UML Modeling Phase:** The software architect creates a **Component Diagram** showing the WiFi Module , UI Display Module ,and HVAC Control Module and how they connect via specific software APIs. Later, the lead programmer creates a **Class Diagram** for the HVAC Control Module , defining classes like RelayController , TemperaturePID , and Scheduler , detailing their specific functions and variables.
- Debugging Phase:**
 - While bringing up the custom PCB, the I2C temperature sensor isn't responding. The engineer uses a J-Link debugger (**On-board Debugging**) to halt the processor, read the I2C status register, and realizes an arbitration-loss error flag is set.
 - Later, the RTOS is implemented. The UI starts lagging heavily whenever the WiFi module transmits. Halting the CPU doesn't help. The engineer uses Segger SystemView (**Task-level Debugging**) to trace RTOS events. They discover the WiFi task (highest priority) is consuming 90% of the CPU bandwidth in a busy-wait loop, starving the UI task (lower priority). They fix this by replacing the busy-wait with an RTOS event flag.

6. Advantages and Limitations

Comparison: Class vs. Component Diagrams

Feature	Class Diagram	Component Diagram
Abstraction Level	Low (Logical/Code level).	High (Architectural/Module level).
Primary Elements	Classes, Attributes, Methods.	Components, Interfaces, Ports.
Best Used For	Defining detailed object structures and inheritance.	Planning system deployment and module dependencies.

Limitation	Becomes unreadable for large, system-wide architectures.	Lacks the detail necessary for actual coding.
------------	--	---

Comparison: On-Board vs. Task-Level Debugging

Feature	On-Board (JTAG/SWD)	Task-Level (Trace/RTOS Aware)
Focus	Hardware registers, memory, assembly instructions.	OS Threads, IPC, CPU utilization, timing.
Impact on System	Halts CPU; destroys real-time temporal behavior.	Non-intrusive (usually); maintains real-time operation.
Best Used For	Initial hardware bring-up, peripheral configuration bugs.	RTOS deadlocks, race conditions, performance profiling.

7. Conclusion

Effective embedded system development requires a structured approach from design to deployment. UML structural modeling using Class and Component diagrams ensures that both the high-level architecture and the low-level code are logically sound before any code is written. When issues inevitably arise, an engineer must possess a dual debugging methodology. On-board debugging is the ultimate tool for resolving fundamental hardware-software interface issues, while task-level debugging is indispensable for analyzing and correcting the complex